

Настройка прав к методам

Описание проблемы

Платформенная система прав позволяет разграничивать права доступа к методам БЛ для разных ролей. Однако если доступ к функционалу ограничен лишь интерфейсно, то остается возможность прямого вызова методов из консоли.

У нас 90 тысяч методов и контролировать, как настроены права доступа к каждому – огромная и очень трудозатратная задача. Разработчики могут существенно помочь, если заранее проанализируют, какие права необходимы их методам, и реализуют их проверку. В конце концов, никто лучше разработчика не знает его функционал.

Небезопасный пример

Пусть есть общедоступный метод БЛ `getUserSalary(user_id)` с одним параметром, который возвращает данные по зарплате для указанного сотрудника. Пусть этот метод корректно зовется из интерфейса (с идентификатором текущего пользователя).

Если метод не накладывает дополнительных ограничений, то злоумышленник имеет возможность получения информации о зарплатах других сотрудников путем обычного вызова с разными идентификаторами.

Последствия эксплуатации

OWASP (международное сообщество безопасников) ставит проблему некорректно реализованного управления доступом на первое место среди угроз безопасности!

Именная эта уязвимость зачастую является входной точкой для злоумышленника и помогает реализовать более сложные и эффективные атаки. Примеры:

- выгрузить документы, которые не должны быть доступны (выгрузка кадровых документов кем-то не из hr);
- можно поменять настройки схемы, включая ее уровень безопасности;
- удалить назначенные встречи в чужом календаре, например, запись к директору:)

Как исправить?

При настройке прав главным принципом безопасности является **принцип минимально-достаточных прав**. Это значит, что права должны быть достаточны для выполнения рабочих задач (в случае методов БЛ – для выполнения бизнес-сценария пользователем), но не более того.

У нас есть две системы прав:

- [полная](#) (модуль PermissionChecker),
- [легкая](#) (модуль LightRight).

[Как определить, какая система прав используется на сервисе](#)

Полная система прав позволяет гибко настраивать права доступа в зависимости от роли сотрудника, тогда как легкая система – позволяет либо разрешить, либо запретить для всех.

Кроме того, можно в принципе не использовать систему прав на сервисе, тогда методы этого сервиса будут доступны всем (так делать мы, конечно, не рекомендуем).

При настройке прав к методу нужно руководствоваться следующими правилами:

1. Предпочтительнее всего **использовать полную систему прав**.

[Подробнее о системе прав](#)

2. **Проверять права на бэкэнде**. Помните, что злоумышленник будет вызывать метод из консоли, а значит сможет передавать в запросе нужные ему параметры и идентификаторы. Если реестр на интерфейсе недоступен и показывает ошибку "нет прав", то и методы, формирующие этот реестр не должны быть доступны для вызова. Или, если какой-то функционал ограничен лицензией, то лицензию недостаточно проверять на фронте – ее нужно проверять и в методах, реализующих этот функционал.
3. Часто разработчики обрабатывают ограничение доступа по участку системы, но забывают **учесть область видимости**.

Например, у пользователя ограничены права на просмотр карточек сотрудников: он может смотреть только сотрудников своего подразделения. В интерфейсе все отрабатывает корректно, но если сделать вызов из консоли,

можно получить данные из карточек сотрудников других подразделений. Так происходит, когда разработчик в коде проверяет, что у пользователя есть доступ к участку "Карточка сотрудника", но не проверяет, какие у пользователя есть ограничения по этому участку.

[Подробнее об ограничениях](#)

[Подробнее об области видимости](#)

4. Реализуйте **межсхемный барьер**. Если ваш сервис хранит данные из разных схем в одной БД, то обязательно нужно проверять из какой схемы сделан запрос и не отдавать данные всех схем или чужой для пользователя схемы.
 - Особое внимание стоит уделить методам, которые принимают в качестве параметра: идентификатор пользователя, идентификатор клиента, персону и т.п.
 - Если вашему методу нужен идентификатор текущего пользователя или клиента, то нужно брать его из сессии.

Другие параметры, влияющие на возможность вызова метода

Доступность метода бизнес-логики

Помимо системы права на возможность вызвать метод влияет его доступность (accessibility). Ошибки при выборе значения этого атрибута возникают довольно часто, потому что один параметр отвечает сразу за два свойства:

- доступность метода для внешнего вызова
- стабильность API

В контексте безопасности нас интересует только доступность метода, возможны следующие варианты:

- **Internal** - метод недоступен для вызова извне и может использоваться только внутри сервиса.
- **Trusted Application Contract** - метод доступен для вызова только из доверенных точек входа.
- Все остальные варианты означают, что метод доступен для вызова извне. В том числе это касается методов, у которых в атрибуте **Accessibility** задано значение "Internal Application Contract".

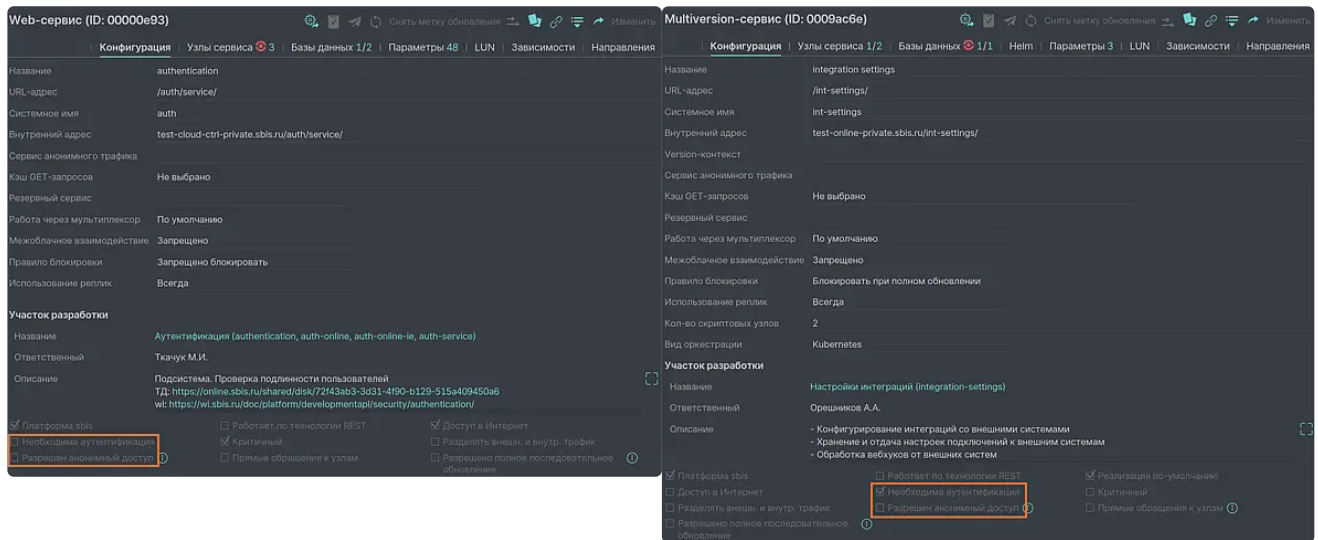
[Подробнее об атрибуте Accessibility](#)

Настройки сервисов

От настроек сервиса также зависит возможность вызова методов. Здесь нас будет интересовать только два параметра:

- Требуется аутентификация
Если у сервиса, где вы пытаетесь вызвать метод, этот параметр установлен, то вызов методов будет возможен, только если у пользователя имеется сессия.
Если параметр не установлен, методы сможет вызвать кто угодно.
- Разрешен анонимный доступ
Если у сервиса включена аутентификация и разрешен анонимный доступ, то методы сможет вызывать пользователь с гостевой сессией.
Если аутентификация включена, но анонимный доступ запрещен, методы смогут вызывать только реальные пользователи, гостевой сессии будет недостаточно.

К примеру, для вызова методов на сервисе **auth** сессия не нужна (иначе бы пользователи не смогли аутентифицироваться). А вот методы сервиса **int-settings** (настройки интеграций) незачем вызывать пользователю без сессии или гостю.



Здесь также стоит заметить, что многие методы присутствуют не на одном сервисе, а сразу на нескольких. И если на каком-то одном сервисе аутентификация необходима, вполне может оказаться, что на другом сервисе ваш метод доступен без аутентификации.

Наличие сервиса на публичных сайтах

Также на возможность вызвать метод влияет наличие сервисов, в которые он входит, на публичных сайтах.

Возможны следующие варианты:

- Сервисы доступны только с серверов, входящих в облако (сегмент private).

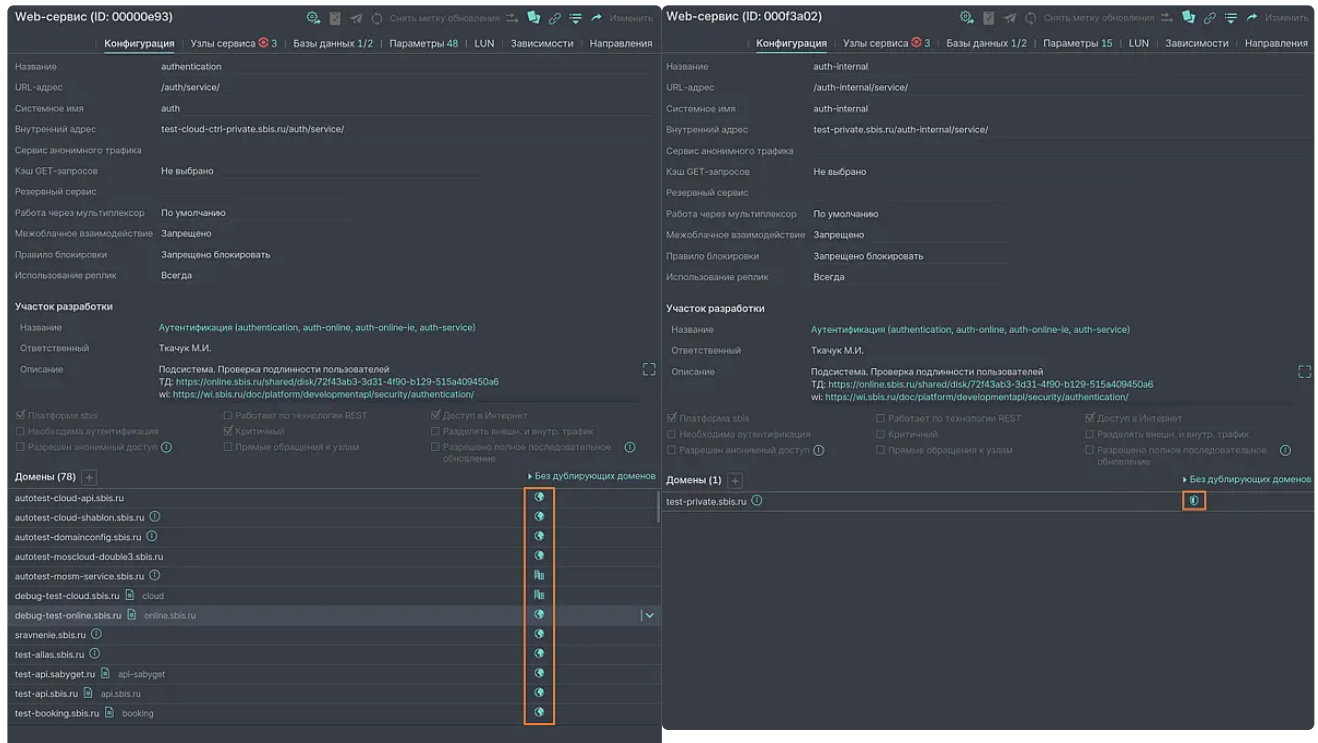
Если ваш метод есть только на таких сервисах, из интернета - или даже из рабочей сети компании его не вызвать.

- Сервисы доступны только из внутренней сети (например, cloud).

Если ваш метод есть только на таких сервисах, его можно будет вызвать из внутренней сети компании, а вот из интернета вызвать не получится.

- Сервисы доступны извне.

Информацию о том, на каких доменах есть каждый конкретный сервис можно посмотреть в облаке. К примеру, сервис **auth** есть на самых различных сайтах, а вот **auth-internal** - только в private сегменте.



Позволю себе еще раз обратить внимание на то, что многие методы присутствуют сразу на нескольких сервисах. И если какой-то из этих сервисов есть на публичном сайте - его методы окажутся доступны из интернета (точнее, здесь уже нужно будет смотреть остальные условия).

Особые замечания

Кабинет физ. лица

Кабинет физического лица (сервис lk.sbis.ru) не предполагает наличия нескольких пользователей, т.к. к этому кабинету ни у кого, кроме конкретного пользователя, доступа нет. Поэтому данный сервис работает **без системы прав**.

Таким образом, если ваш метод присутствует на указанном сервисе, стоит обратить особое внимание на то, каким образом в нем осуществляется разграничение прав.

Наиболее пристально стоит посмотреть на методы, нарушающие межсхемный барьер, принимающие, в качестве одного из параметров идентификаторы пользователя, клиента и т.п.